



Form3 Design Mobile Sales Canvassing Application

GROUP MEMBER:

No.	Student Name	Student ID	Study Program
1.	Dannis Surya Ridfan	001202300188	IT (Cyber Security)
2.	Pieter Utomo Wijaya	001202300096	IT (Cyber Security)
3.	Muhammad Ibnul Aroby	001202300086	IT (Cyber Security)

Advisor : Abdul Ghofir

Submitted for
Capstone Design Project
to Faculty of Computer Science
President University

Table of Contents

A. SYSTEM DESIGN	7
1. System Architecture Overview	7
B. HIERARCHICAL/ITERATIVE DESIGN	11
1. Block Diagrams.....	11
2. Global Variable/Message Parsing Interface.....	15
3. Scrum Methodology to Implement Steps in Software Engineering Design	15
4. Component References & Libraries Used	16
5. UML Diagrams.....	17
C. STANDARDS USED	20
D. IMPLEMENTATION AND TESTING SCENARIO	24
1. Implementation Phases	24
2. Testing Scenarios	25
REFERENCES	33

STATEMENT OF ORIGINALITY

In my capacity as an active student at President University and as the author of the Capstone Design Project stated below:

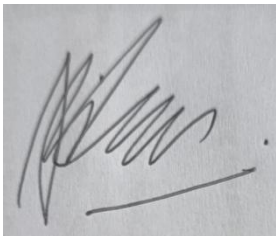
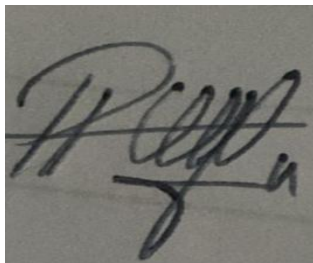
Name : 1. Dannis Surya Ridfan – 001202300188
2. Pieter Utomo Wijaya – 001202300096
3. Muhammad Ibnul Aroby – 001202300086

Faculty : Computer Science

I hereby declare that my Capstone Design Project entitled “**TITLE**” is to the best of my knowledge and belief, an original piece of work based on sound academic principles. If there is any plagiarism detected in this final project, I am willing to be personally responsible for the consequences of these acts of plagiarism and will accept the sanctions against these acts in accordance with the rules and policies of President University.

I also declare that this work, either in whole or in part, has not been submitted to another university to obtain a degree.

Cikarang, 1 April 2025

Signer 1	Signer 2	Signer 3
		
Dannis Surya Ridfan 001202300188	Pieter Utomo Wijaya 001202300096	Muhammad Ibnul Aroby 001202300086

Mobile Sales Canvassing Application

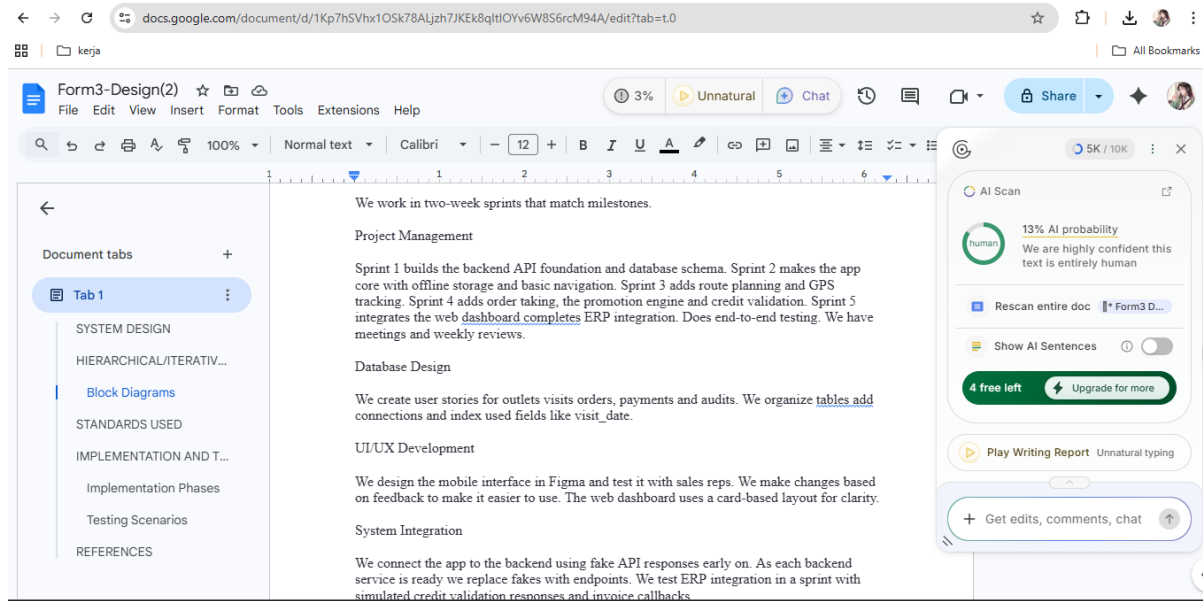
Approved:

Abdul Ghofir
Capstone Advisor

{Head of Study Program Name}
Program Head of XX

Prof. Dr. Ir. Wiranto Herry Utomo, M.Kom.
Dean of Faculty of Computer Scien

SCREENSHOT OF ZEROGPT



PART 3 DESIGN

Consists of:

- A. SYSTEM DESIGN**
- B. HIERARCHICAL / ITERATIVE DESIGN**
- C. COMPONENT COST ANALYSIS**
- D. FUNCTIONAL TESTING**

A. SYSTEM DESIGN

The Mobile Sales Canvassing Application follows a client-server architecture with offline capabilities. The system consists of a mobile client (Flutter), a backend server (Node.js + Express), a relational database (PostgreSQL), and an optional caching layer (Redis). External integration includes the company's ERP system via REST API.

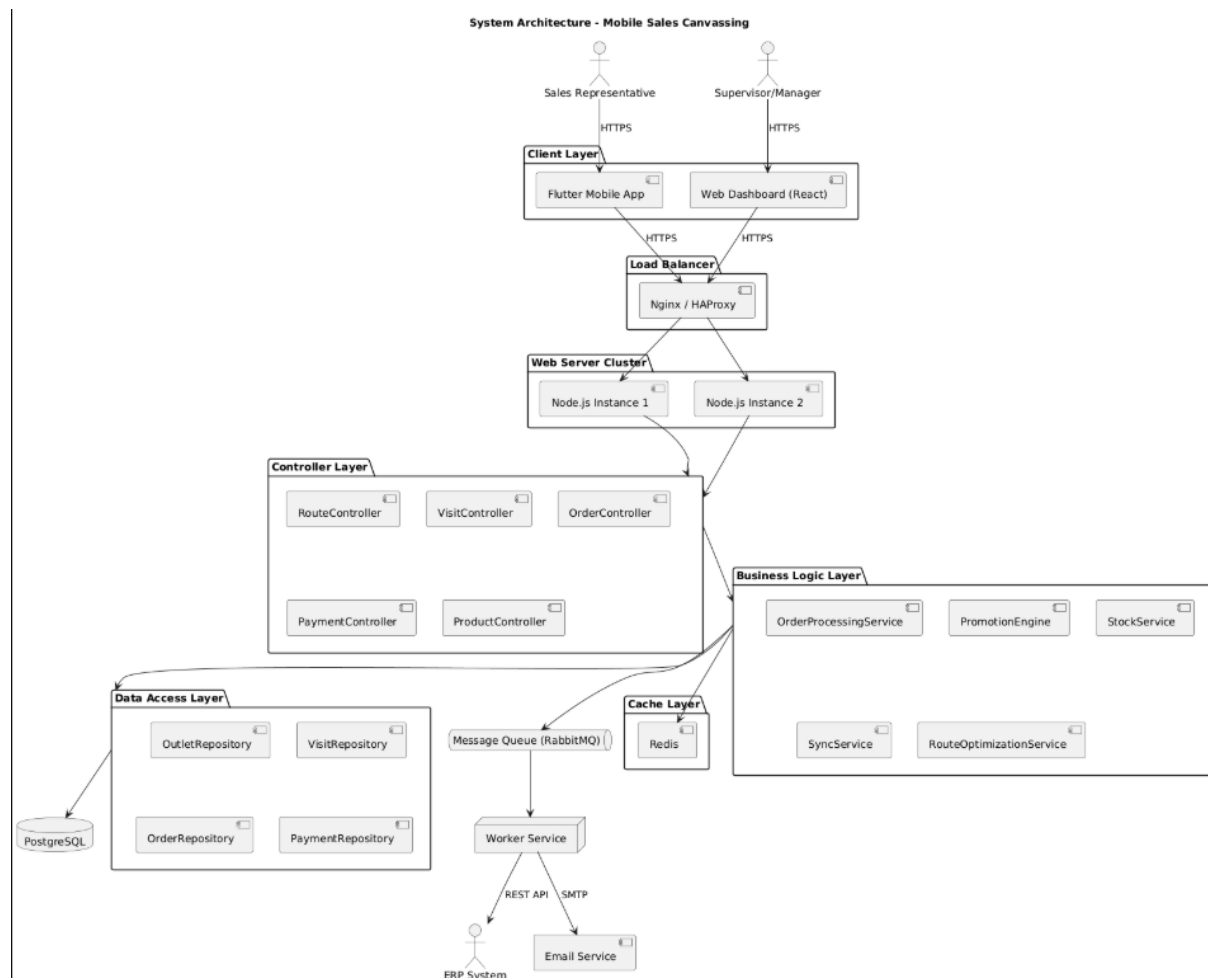


Figure 1 System Architecture

1. System Architecture Overview

The architecture is made up of layers that take care of different things.

- Mobile Client

The mobile client is used by salespeople on their Android cellphones. They use a Flutter application that saves data locally in SQLite when the cellphone is not connected to the internet. This application uses the geolocator package to record GPS locations and flutter_map to display OpenStreetMap maps. The mobile client can handle order entry, payment recording and merchandising audits. It talks to the backend using REST API calls over HTTPS. When the cellphone gets connected to the internet again it synchronizes the data that was saved locally.

- Web Dashboard

Managers and supervisors can use a web browser to visit a dashboard that is built on React. This dashboard shows them results, visit coverage, sales performance metrics and field team positions in real time. The web dashboard and the mobile client use the REST API endpoints.

- Backend Server

The backend server runs Node.js with Express. It takes care of all the HTTP requests. The server has a controller layer that checks the input parameters and sends each request to the service. The business logic layer has modules for order processing, promotion calculation, stock management, route optimization and offline data synchronization. The data access layer uses postgres to execute SQL queries against the PostgreSQL database. The server also talks to the company's ERP system using REST API calls to check credit limits. Send confirmed orders for invoice generation.

- Database

The database uses PostgreSQL to store all the data. The tables in the database include users, employees, sections, outlets, visits, products, units sales orders order details, payments, merchandising audits and promotions. The database uses keys to make sure the data is consistent. It also uses indexes to speed up queries on fields like visit date and outlet identifier.

- Cache Layer

The cache layer uses Redis to store data that is used a lot, like the product catalog, outlet details and active user sessions. This helps reduce the load on the database and makes the system respond faster to requests that are made a lot.

- Message. Worker Service

The server uses a RabbitMQ queue to send messages for tasks that do not need to be done away like sending email notifications or generating weekly reports. A separate worker service takes care of these messages. Does the tasks. This helps keep the web server running smoothly even when there is a lot of work to do.

- Load Balancer and Web Server Cluster

The system uses a load balancer to distribute requests across several Node.js instances. This makes sure the system can handle a lot of users at the time. If one

instance fails the others can keep handling requests. This design makes it easy to add instances as the number of users grows.

- External Integration
- The system talks to the existing ERP system using REST API. It checks the credit limit before an order is submitted and sends the completed orders for invoicing and stock updates. The ERP system also provides the system with real time stock levels for products.

The mobile client and the backend talk to each other using HTTPS with JSON payloads. The system can work offline: when the cellphone is not connected to the internet the mobile app stores data, in a SQLite database and synchronizes it automatically when it gets connected again. The mobile client and the backend server both use the system architecture to make sure everything works smoothly.

B. HIERARCHICAL/ITERATIVE DESIGN

1. Block Diagrams

1. Block Diagram Level 0

The top level diagram shows external actors and the main system boundary.

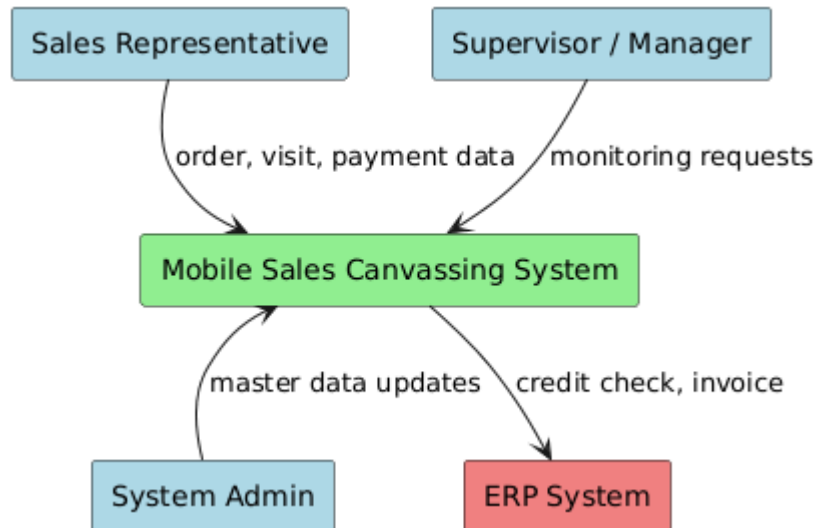


Figure 2.1.1 Block Diagram level 0

Interface Information:

- Sales representatives interact via the mobile app over HTTPS.
- Supervisors and admins access the web dashboard via browsers.
- The backend communicates with the ERP system through REST API (JSON) over HTTPS.
- The mobile app uses a local SQLite database when offline and syncs via REST when online.

2. Block Diagram Level 1 – System Overview

The system decomposes into major subsystems: Mobile Client, Backend Server, Database, Cache, External ERP, and Admin Web.

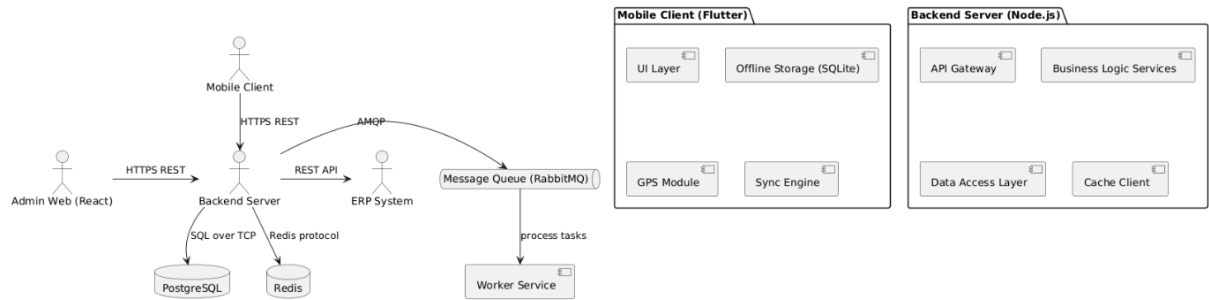


Figure 2.1.2 Block Diagram Level 1

Interface Information:

- Mobile client ↔ Backend: JSON over HTTPS.
- Backend ↔ PostgreSQL: SQL queries via node Postgres.
- Backend ↔ Redis: Redis protocol for caching.
- Backend ↔ ERP: REST API with JSON payloads.
- Web dashboard ↔ Backend: same REST API.

3. Block Diagram Level 2 – Backend Server Subsystem (Node.js)

We focus on the internal architecture of the backend server.

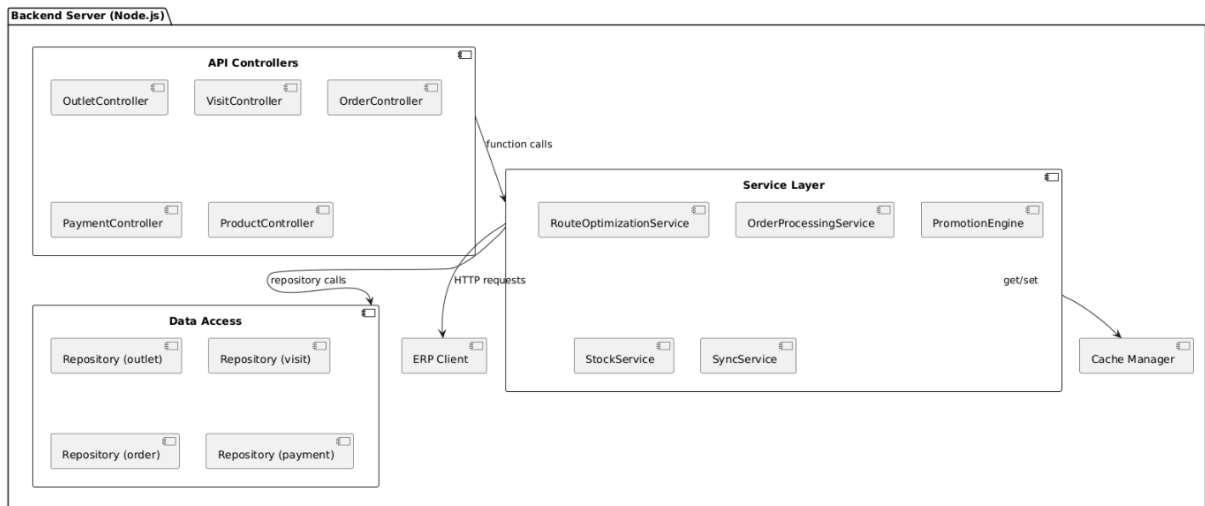


Figure 2.1.3 Block Diagram Level 2

Interface Information:

- Controllers receive HTTP requests, validate parameters, and call services.
- Services contain business logic (e.g., applying promotions, calculating stock).

- Data Access repositories encapsulate SQL queries.
- Cache Manager interacts with Redis for frequently accessed data (e.g., product catalog).
- ERP Client handles communication with the external ERP.

4. Block Diagram Level 3 – Component & Function Level (Order Taking Module)

We focus on the order taking flow to show primitive functions. The sales representative taps the order button in the mobile app. The app sends an HTTP POST to the backend order service. The order service calls the ERP API to validate the customer's credit. After receiving a positive response, it inserts the order into the PostgreSQL database and returns a success status to the app.

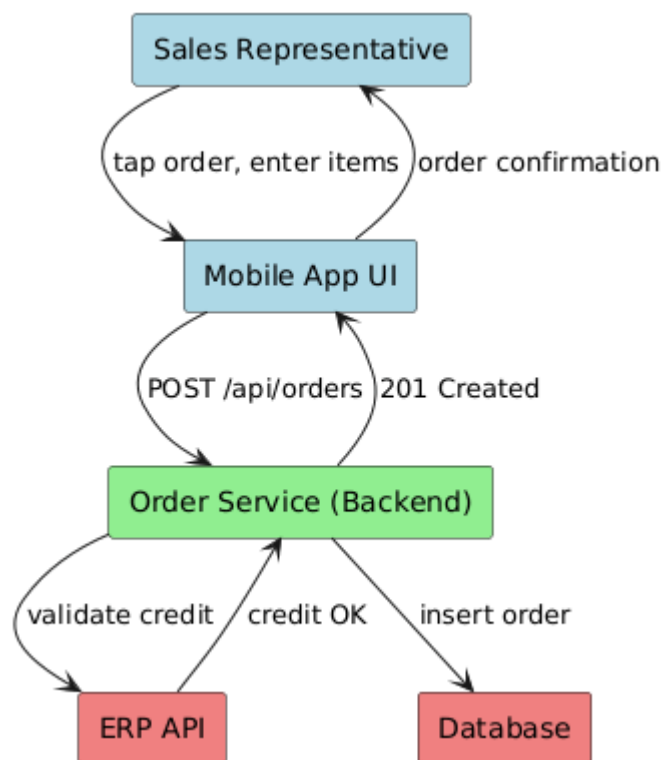


Figure 2.1.4 Block Diagram Level 3 (Order taking module)

Interface Information:

- Data Transfer Object (DTO) OrderDto passes order items, customer ID, and outlet ID from client to controller.
- The service returns an order ID and success status.

- The ERP client uses JSON over HTTPS to query credit limits.
- All validation errors are returned as HTTP 400 with error messages.

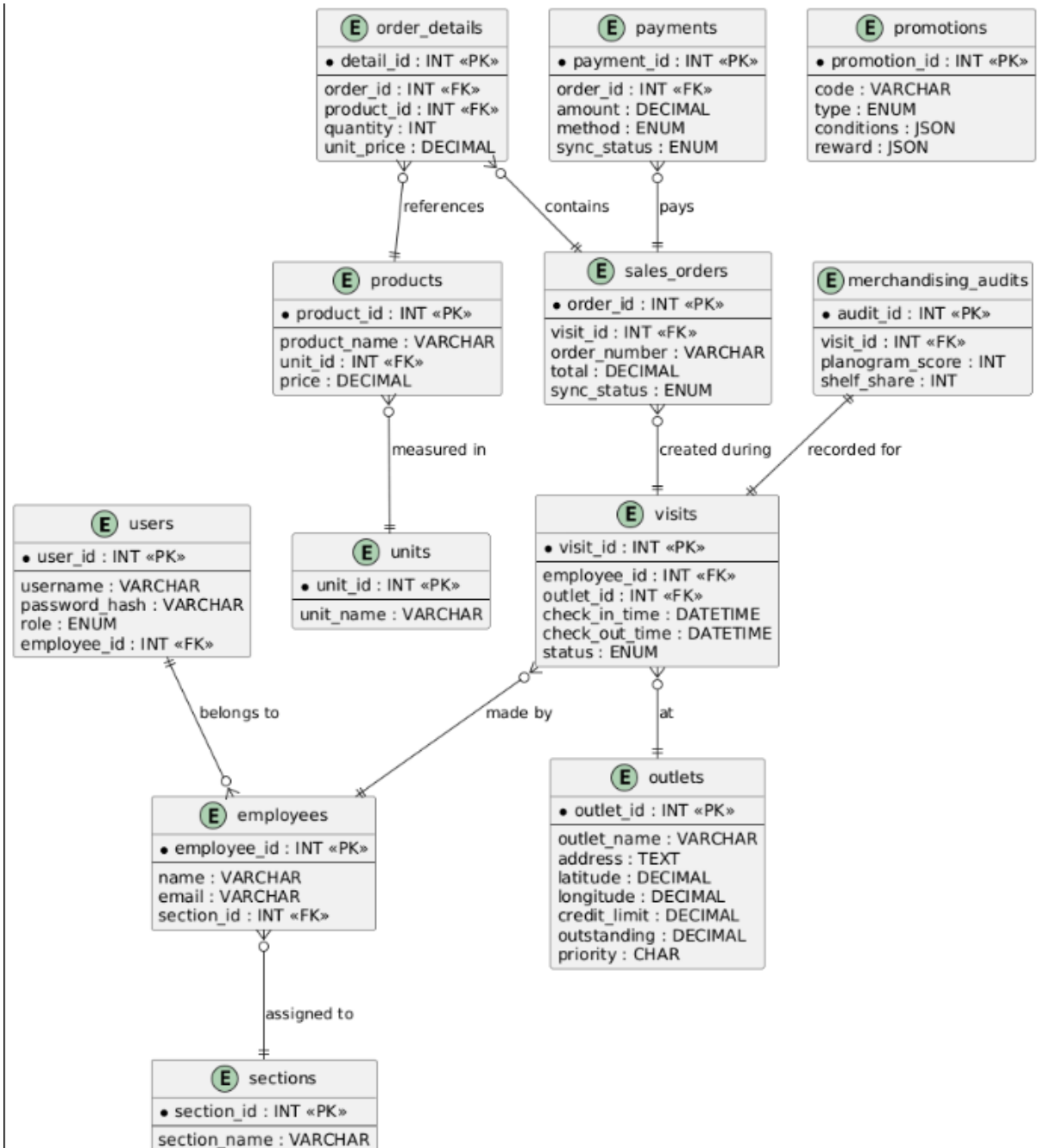


Figure 2.5.1 ERD

2. Global Variable/Message Parsing Interface

The system uses three ways to share data between layers.

- **Parameter Passing**

The mobile app sends data to the backend as JSON objects. For example an order object has the outlet ID, a list of items with product IDs and quantities and the payment method. The controller checks the object. Passes it to the service layer as a simple JavaScript object.

- **REST API Messages**

The app and server talk to each other using HTTP with JSON data. The app calls endpoints like POST /api/visits/checkin with GPS coordinates and outlet ID and POST /api/orders with order data. The backend sends back JSON responses. For ERP integration the backend sends a credit validation request. Gets a JSON response saying yes or no. How much credit is left.

- **Asynchronous Message Queue**

For tasks that can wait the backend sends messages to a RabbitMQ queue. Each message has a task type and parameters. A worker service takes these messages. Processes them on its own keeping the main application fast.

3. Scrum Methodology to Implement Steps in Software Engineering Design

We work in two-week sprints that match milestones.

- **Project Management**

Sprint 1 builds the backend API foundation and database schema. Sprint 2 makes the app core with offline storage and basic navigation. Sprint 3 adds route planning and GPS tracking. Sprint 4 adds order taking, the promotion engine and credit validation. Sprint 5 integrates the web dashboard completes ERP integration. Does end-to-end testing. We have meetings and weekly reviews.

- **Database Design**

We create user stories for outlets visits orders, payments and audits. We organize tables add connections and index used fields like visit_date.

- **UI/UX Development**

We design the mobile interface in Figma and test it with sales reps. We make changes based on feedback to make it easier to use. The web dashboard uses a card-based layout for clarity.

- **System Integration**

We connect the app to the backend using fake API responses early on. As each backend service is ready we replace fakes with endpoints. We test ERP integration in a sprint with simulated credit validation responses and invoice callbacks.

4. Component References & Libraries Used

Hardware

We use laptops with least 8 GB RAM and SSD storage. We use Android smartphones (Android 10 or newer) for testing. We use USB cables for debugging.

Software & Frameworks

- Flutter with flutter_map and geolocator for mapping and GPS.
- Node.js with Express for the backend.
- PostgreSQL for storing data.
- Redis for caching used data.
- React with TypeScript, Tailwind CSS and Recharts for the web dashboard.
- *. Github for version control.

External Services

- ERP REST API for credit validation and invoice generation.

- OpenStreetMap tiles for free map display.

5. UML Diagrams

We use UML to model structure and behavior.

A. Entity Relationship Diagram (ERD)

The database stores data, for users, employees, sections, outlets visits orders order details, products, units, payments, merchandising audits and promotions.

A. Use Case Diagram Actors:

Sales Representative, Supervisor/Manager, ERP System.

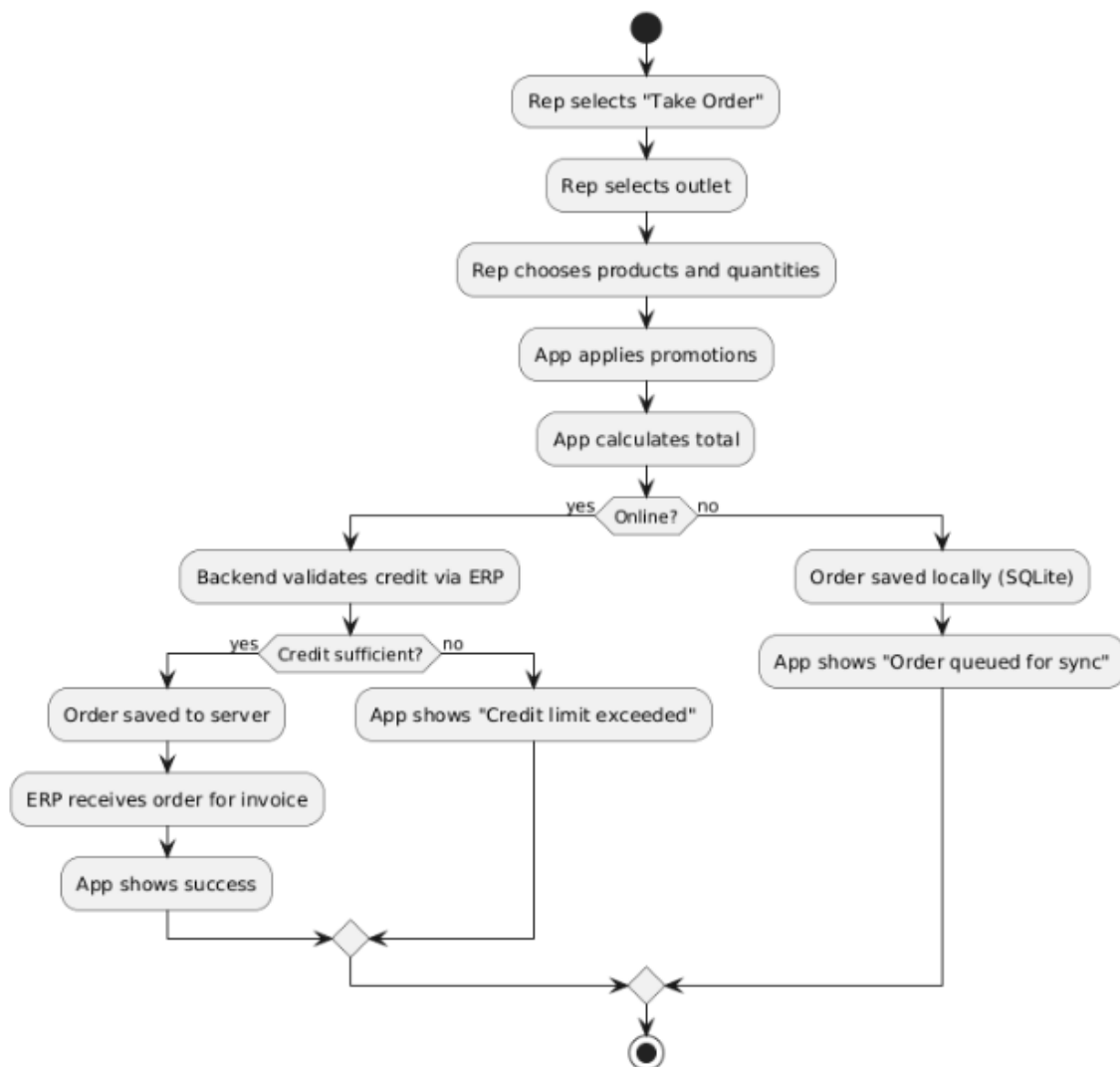


Figure 2.5.2 Use Case Diagram

B. Activity Diagram – Take Order

This diagram models the flow for creating a sales order, including online credit validation and offline fallback.

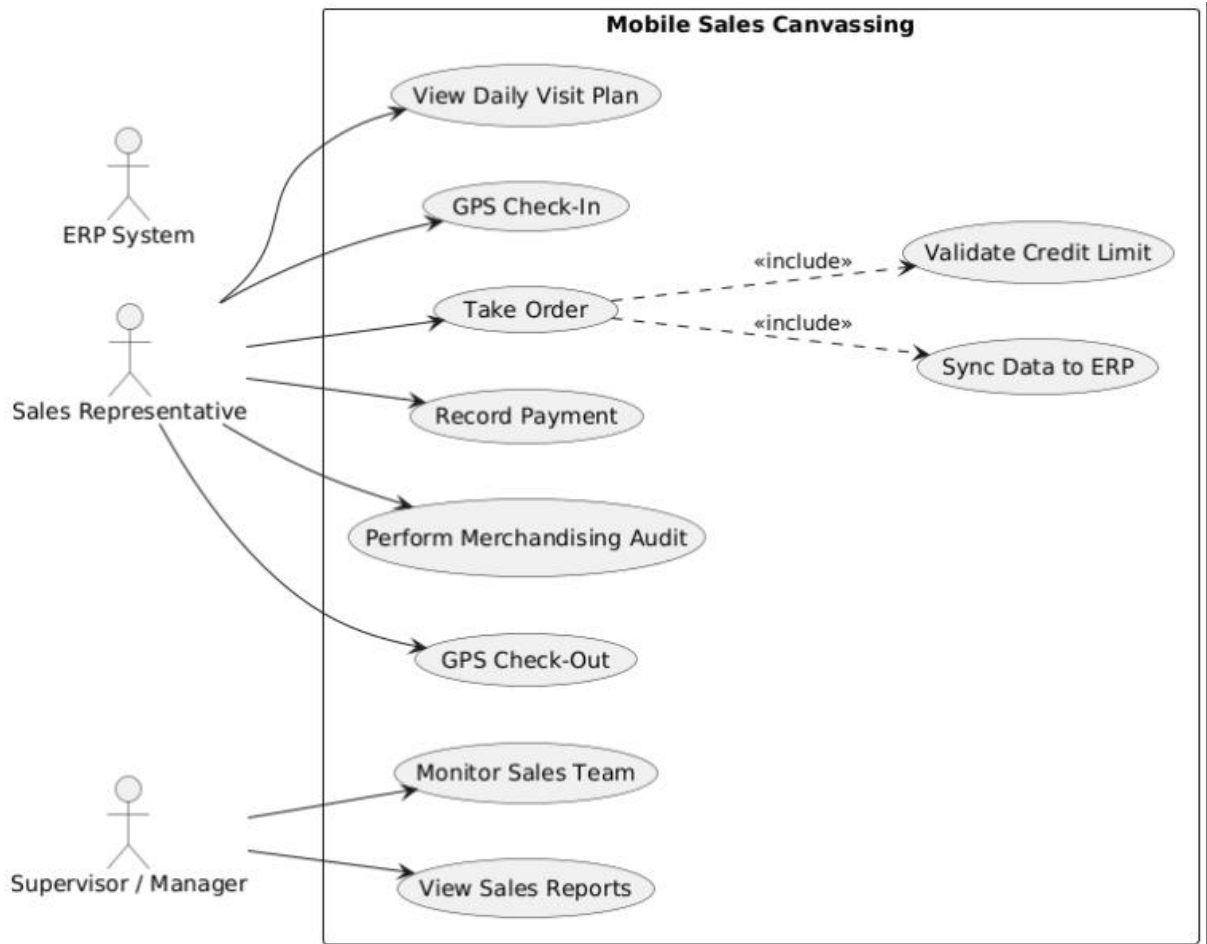


Figure 2.5.3 Activity Diagram – Take Order

C. Sequence Diagram – GPS Check-In

This diagram shows the interaction sequence for validating a check-in using GPS and geofencing.

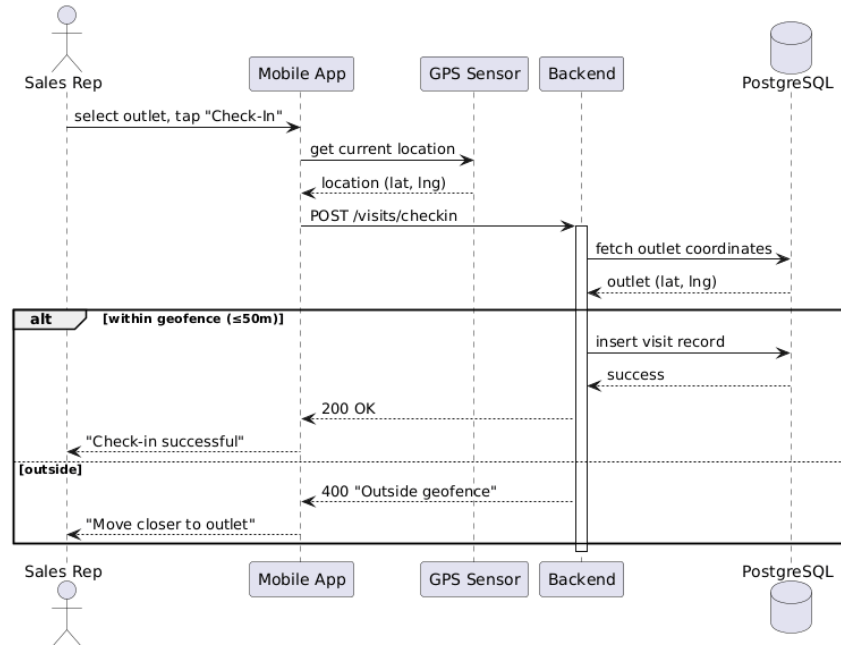


Figure 2.5.4 Sequence Diagram – GPS Check-In

D. Class Diagram

The class diagram shows the main domain classes and their relationships.

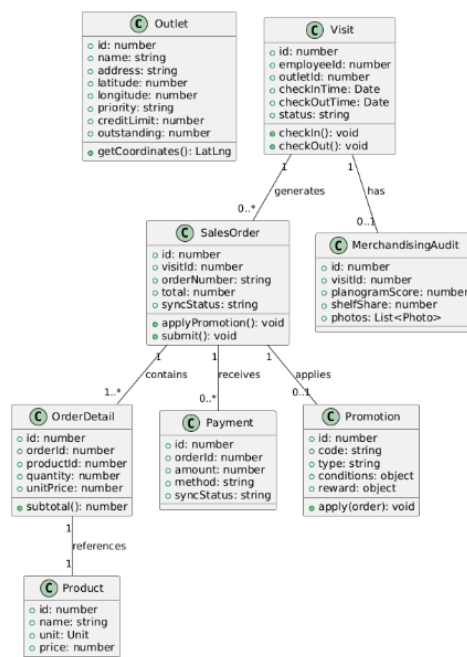


Figure 2.5.5 Class Diagram

C. STANDARDS USED

1. We adopt the following standards to ensure consistency, interoperability, and maintainability across the system.

Category	Standard Technology	Description
Data Format	JSON (RFC 8259)	All API requests and responses between the mobile client, web dashboard, and backend use JSON. The ERP integration also exchanges JSON payloads for credit validation and invoice transmission.
System Modeling	UML 2.5	We use Unified Modeling Language 2.5 for all structural and behavioral diagrams: class diagrams, use case diagrams, activity diagrams, sequence diagrams, and entity relationship diagrams.
API Architecture	REST over HTTPS	The mobile app and web dashboard communicate with the backend through RESTful endpoints. Each resource (outlets, visits, orders, payments) has a clear URI and uses standard HTTP methods (GET, POST, PUT, DELETE). All communication is encrypted with TLS 1.2 or higher.
Offline Storage	SQLite (with JSON serialization)	The Flutter app stores pending transactions and master data in an SQLite database. Records

Category	Standard Technology	Description
		awaiting synchronization are serialized as JSON to maintain compatibility with the backend schema.
Map Data	OpenStreetMap Tile Usage Policy	The flutter_map component requests map tiles from the OpenStreetMap tile server. We follow the tile usage policy: proper attribution, no bulk downloading, and use of a valid user agent string.
GPS Data	NMEA 0183 (via device API)	The geolocator package returns location data compliant with NMEA standards, including latitude, longitude, speed, and accuracy. The system uses the location data for geofencing and route tracking.
Database Access	Parameterized SQL (node-postgres)	All SQL queries from the backend to PostgreSQL use parameterized statements to prevent SQL injection. We use the node-postgres library with prepared statements for all data operations.
Caching	Redis Serialization Protocol	Redis stores serialized JSON strings with configurable time-to-live. Common data such as product catalog, outlet details, and active user

Category	Standard Technology	Description
		sessions are cached to reduce database load and improve response times.
Authentication	JWT (RFC 7519)	The backend issues JSON Web Tokens after successful login. Tokens contain the user ID, role, and expiration time (24 hours). The client includes the token in the Authorization header for each request.
Authorization	Role-Based Access Control (RBAC)	Access to resources is restricted based on user roles (admin, manager, supervisor, rep). Middleware enforces permissions at the API level.
Device Binding	Device ID with JWT claim	The system binds a user account to a specific device by storing a device fingerprint during login. If a login attempt occurs from an unrecognized device, the user must verify via OTP.
Data Security at Rest	SQLCipher for local database	The offline SQLite database on the mobile device is encrypted using SQLCipher, preventing unauthorized access to sensitive data if the device is lost or stolen.

Category	Standard Technology	Description
Tamper Detection	Mock GPS detection	The mobile app detects if the device has mock location enabled and rejects check-in attempts. It also validates GPS accuracy to ensure location authenticity.
Audit Logging	Centralized logging with Winston	All critical actions (login, order creation, payment recording, data sync) are logged with timestamps, user ID, and IP address. Logs are stored in the database and can be reviewed by administrators.
Version Control	Git	We manage all source code with Git. The repository is hosted on GitHub, and we use conventional commit messages for traceability.
Development Methodology	Scrum	We follow the Scrum framework for iterative development. Each sprint lasts two weeks and delivers a working increment of the software.

D. IMPLEMENTATION AND TESTING SCENARIO

We implement the system in five sprints, each focusing on a set of features. Testing occurs throughout the development cycle, with dedicated test cases for each module.

1. Implementation Phases

- **Sprint 1: Backend Foundation**

We set up the Node.js + Express server, create the PostgreSQL database schema, and implement basic CRUD operations for outlets, products, and users. We also set up the Redis cache, the RabbitMQ message queue, and implement JWT authentication with role-based access control.

- **Sprint 2: Mobile Foundation**

We create the Flutter project, configure offline storage with SQLCipher (encrypted SQLite), and implement basic navigation, login, and GPS permission handling. We integrate the flutter_map and geolocator packages and add mock location detection.

- **Sprint 3: Route Planning and GPS Tracking**

We implement the daily visit plan generation, route optimization service, GPS check-in/check-out with geofencing, and store visit records. We also build the real-time location tracking for the web dashboard and add device binding for security.

- **Sprint 4: Order Taking, Promotion Engine, and Payments**

We implement the order creation flow, including product selection, quantity input, automatic promotion calculation, credit validation via ERP, and offline queuing. We add the payment recording module, merchandising audit, and audit logging for all critical actions.

- **Sprint 5: Web Dashboard, Security Hardening, and Final Integration**

We build the React web dashboard with real-time maps, sales performance charts, and reports.

We complete ERP integration (credit validation, invoice transmission), add centralized audit log viewing, perform end-to-end testing, and conduct security reviews (penetration testing for API endpoints).

2. Testing Scenarios

The following table lists test cases for each major feature, including security aspects. Each row describes the implementation approach, the test procedure, alternative scenarios, and the expected outcome.

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
1	User Authentication with JWT	The backend validates email/password and returns a JWT. The mobile app stores the token and includes it in subsequent requests.	Enter valid credentials and click Login.	Wrong password; non-existent email; inactive account; missing fields.	Successful login redirects to the dashboard. Invalid credentials display an error message. Token expires after 24 hours; renewal is handled via refresh token.
2	Role-Based Access Control	Middleware on the backend checks the	Log in as a rep and try to access a supervisor-only	A supervisor accesses a rep-only	Access denied with HTTP 403. The correct

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
		user's role before allowing access to specific endpoints.	endpoint (e.g., /api/reports).	endpoint; a rep attempts to modify master data.	role can access authorized endpoints.
3	Device Binding	During login, the backend records a device fingerprint (Android ID or unique identifier). Subsequent logins from a different device require OTP verification.	Log in from a new device after registering the first device.	User loses phone and tries to log in from another device; attacker attempts to use stolen credentials from an unknown device.	Login from registered device succeeds. Login from new device triggers OTP request; after OTP verification, the new device is registered.
4	GPS Check-In with Geofencing	The app obtains current GPS coordinates and sends them with the outlet ID to the backend. The backend calculates the distance from the outlet's stored coordinates and, if within 50 meters, records the visit.	Select an outlet, stand within the geofence, and tap Check-In.	GPS off; outside geofence; no internet connection; mock location enabled.	Check-in recorded successfully when inside. Outside geofence shows "Move closer". Offline stores check-in locally and syncs later. If mock location detected, check-in is rejected with a warning.

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
5	Offline Order Taking with Encryption	When offline, the app saves orders in the local SQLCipher database with sync_status = 'Pending'. Data is encrypted at rest.	Turn off internet, create an order, and submit.	Device loses connection mid-order; order contains invalid product; duplicate submission.	Order saved locally with pending status. After reconnection, sync occurs automatically and order appears in the backend. Local database cannot be read without the encryption key.
6	Promotion Application	The backend promotion engine applies rules based on quantity thresholds (e.g., “Buy 10 get 5%”).	Add 10 units of a product that has an active quantity-based promotion.	Promotion expired; minimum quantity not met; promotion type not applicable.	Discount calculated correctly and reflected in the order total. Expired or inapplicable promotions do not apply.
7	Credit Validation with ERP	When a rep submits an order, the backend calls the ERP REST API to check the	Submit an order for an outlet with sufficient credit.	ERP service unavailable; credit limit exceeded; ERP	Order proceeds if credit OK. Exceeded credit shows an error

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
		outlet's credit limit. The ERP returns approval or rejection.		returns unexpected data.	message. If ERP is down, the order is queued for later validation and a notification is sent to the supervisor.
8	Real-time Monitoring on Web Dashboard	The backend stores the latest GPS positions of active reps. The web dashboard fetches these positions every 30 seconds via REST API and displays them on a map.	Log in as supervisor and open the live map.	No rep has checked in; GPS data missing; backend service restart.	Map shows markers for reps with recent location. If no data, a message indicates no active reps.
9	Sync Conflict Resolution	An offline order is created while a product's price is updated online. Upon sync, the system uses the current price and flags the order for manual review if	Create an order offline; change the product price on the server; then sync.	Duplicate order submitted accidentally; the same order is synced twice.	The order uses the current price; the system logs a warning and notifies the supervisor. Duplicates are prevented by checking unique

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
		there is a significant discrepancy.			order numbers.
10	Data Integrity – Concurrent Visits	Two reps attempt check-in at the same outlet simultaneously. The backend uses database transactions to ensure both records are inserted.	Simulate two concurrent requests to the same outlet using a load-testing tool.	Database lock; transaction rollback.	Both visits are recorded with distinct IDs; no data loss or corruption.
11	Merchandising Audit Submission	The rep fills a checklist, captures a photo, and submits the audit. The system stores the data and links it to the visit.	Fill the audit form, take a photo, and submit.	Missing mandatory fields; photo exceeds file size limit; invalid file type.	Data saved successfully. Validation errors are displayed for missing fields or invalid files.
12	Payment Recording	The rep records a partial payment. The system updates the outlet's outstanding balance and marks the order as partially paid.	Enter a payment amount less than the total, select payment method, and optionally upload a receipt.	Payment amount exceeds remaining balance; receipt file too large; payment method not selected.	Outstanding balance reduced correctly. Errors displayed for invalid inputs.

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
13	Daily Visit Plan Generation	The backend automatically generates a daily visit plan for each rep based on assigned outlets, visit frequency, and priority (A/B/C).	Log in as rep and view the plan.	No outlets assigned; manual override by supervisor.	The plan lists outlets in optimized order. If no outlets, a message indicates no assignments.
14	Route Optimization	The backend uses the Google Maps Distance Matrix API (or a similar service) to order outlets by travel time.	Request route optimization for a set of outlets.	API rate limit exceeded; invalid coordinates.	The outlets are ordered to minimize travel distance. If the API fails, the system falls back to a simpler heuristic.
15	Audit Logging	All critical actions (login, order creation, payment, data sync) are logged with user ID, timestamp, and IP address. The logs are stored in the database.	Perform a login, create an order, and record a payment. Then view the audit log via the admin dashboard.	Log database fails; log entry is missed due to high concurrency.	All actions are recorded with correct details. If logging fails, the system retries and raises an alert.
16	Data Encryption	The SQLCipher library	Try to read the local database file with a	Device is lost or stolen;	The database cannot be

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
	at Rest (Local)	encrypts the offline SQLite database with a key derived from the user's password.	SQLite browser without the encryption key.	attacker attempts to extract data.	opened; only the app with the correct key can access it.
17	Mock GPS Detection	The app checks for mock location providers and prevents check-in if mock location is enabled.	Enable mock location in developer options, then attempt check-in.	Device uses a fake GPS app; GPS signal is spoofed.	Check-in is rejected with an error message: "Mock location detected. Please disable fake GPS."
18	Session Timeout	The backend sets a JWT expiration of 24 hours. After expiration, the user must re-authenticate.	Stay logged in for 24 hours without activity; then perform an action.	User is inactive for a short period but within token lifetime.	After 24 hours, the token is rejected and the user is redirected to the login page.
19	Offline Sync Security	Pending sync data is stored in the encrypted local database. When syncing, the app sends the data over HTTPS.	Create an order offline, then reconnect to the internet.	An attacker intercepts the sync request (requires breaking TLS).	Data is transmitted securely; the backend validates the JWT and accepts only authenticated requests.

No.	Module / Feature	Implementation Scenario	Testing Procedure	Alternative / Exception Scenario	Expected Result
20	ERP Integration Security	The backend stores ERP API credentials as environment variables, not in code. API calls use API keys or OAuth2 tokens.	Perform a credit validation request that triggers an ERP call.	ERP credentials are leaked; an attacker tries to call the ERP directly.	Credentials are not exposed in logs or client responses. Only the backend can make ERP calls.

REFERENCES

- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Fielding, R. T. (2000). Architectural Styles and the Design of Network-Based Software Architectures (Doctoral dissertation, University of California, Irvine).